

E-Commerce Admin & Order Management

Name	Himanshu Gautam
Institute	AKS University Satna
Company	TCS iON Digital Learning
Industry Project Title	E-Commerce Admin & Order Management
Start Date	08 Apr 2026
End Date	27 May 2026
Total Effort (hrs)	90 hr
Enviroment	Windows 11, Visual Studio Code, Localhost (Django Development Server)
Tools Used	React.js, Django, Django REST Framework, PostgreSQL, Razorpay, Postman, Git, GitHub, Visual Studio Code

Acknowledgements

The successful completion of this project would not have been possible without the support and resources provided by **AKS University, Satna**. I am grateful to the University and the Department of Computer Science & Engineering for giving me the opportunity to work on this project and apply the knowledge gained during my academic studies.

Working on the **E-Commerce Admin & Order Management System** has been a valuable learning experience. This project helped me understand the practical aspects of web application development, database management, API integration, inventory management, order processing, and payment gateway implementation.

I would also like to acknowledge the contribution of various online learning platforms, technical documentation, open-source communities, and development tools that provided guidance throughout the project development process. Their resources helped me overcome challenges and improve my technical skills.

Finally, I would like to thank everyone who directly or indirectly supported me during the development of this project. Their encouragement and assistance played an important role in the successful completion of this work.

Objective and Scope

The primary objective of this project is to develop a centralized **E-Commerce Admin & Order Management System** that automates and streamlines the management of products, inventory, orders, payments, and business operations. The system is designed to provide administrators with a single platform to efficiently monitor and control all e-commerce activities, reduce manual effort, improve operational accuracy, and enhance overall business productivity.

The application enables administrators to manage products and categories, track inventory levels in real time, process customer orders, handle payment transactions, monitor delivery status, and generate business reports through an interactive and user-friendly interface. By integrating modern web technologies and secure payment services, the system ensures efficient business management and a better operational workflow.

The scope of this project covers the following areas:

- Secure administrator authentication and access management
- Product and category management with add, update, delete, and image upload functionalities
- Inventory management with real-time stock monitoring and automatic stock updates
- Customer order processing and complete order lifecycle management
- Order status tracking including Pending, Confirmed, Shipped, Delivered, and Cancelled states
- Secure payment gateway integration using Razorpay for online transactions
- Automatic payment verification and transaction management
- Centralized dashboard for monitoring products, orders, inventory, and sales activities
- Sales reporting and analytics for business performance evaluation
- Database management for products, orders, customers, inventory, and payment records
- REST API-based communication between React frontend and Django backend

- Responsive web interface for efficient administration and management
- Business insights through order statistics, revenue reports, and sales analysis
- Scalable architecture that can be extended with customer-facing shopping features, mobile applications, AI-based recommendations, and multi-vendor support in the future

The project provides a practical implementation of modern full-stack web development technologies and demonstrates how e-commerce business operations can be effectively automated through a centralized management system.

Problem Statement

In today's digital era, e-commerce businesses handle a large number of products, customer orders, inventory records, and payment transactions on a daily basis. Managing these operations manually or through separate systems often creates difficulties in maintaining accuracy, efficiency, and business growth. As the number of customers and transactions increases, traditional management methods become less effective and more prone to errors.

The following challenges are commonly faced in e-commerce administration:

- **Manual Inventory Tracking:** Maintaining stock records manually can lead to inaccurate inventory information, stock mismatches, and product availability issues.
- **Delayed Order Processing:** Administrators need to manually monitor and update order statuses, which increases processing time and affects customer satisfaction.
- **Order Management Complexity:** Handling multiple orders simultaneously becomes difficult without a centralized management system.
- **Payment Management Issues:** Tracking payment confirmations and transaction records manually may result in errors and inconsistencies.
- **Lack of Real-Time Monitoring:** Business owners often face difficulties in tracking sales performance, inventory levels, and order activities in real time.
- **Data Redundancy and Human Errors:** Repeated manual data entry increases the chances of incorrect information, duplicate records, and operational mistakes.
- **Limited Reporting and Analytics:** Traditional systems provide limited business insights, making it difficult to analyze sales trends, revenue growth, and overall performance.

- **Scalability Challenges:** As the business grows, manual processes and disconnected systems become difficult to manage efficiently.

There is a clear need for a centralized and automated system that can efficiently manage products, inventory, orders, payments, and reporting activities from a single platform. The **E-Commerce Admin & Order Management System** addresses these challenges by providing a secure, scalable, and user-friendly solution that improves operational efficiency, reduces manual effort, minimizes errors, and supports better business decision-making.

Existing Approaches

Several approaches are currently used by businesses to manage e-commerce operations and online store administration. These solutions help organizations handle products, inventory, orders, and payments, but many of them have limitations in terms of flexibility, scalability, customization, and cost.

Manual Management Systems

Many small businesses still manage products, inventory, and customer orders using spreadsheets, paper records, or basic software tools. While this approach is simple and inexpensive initially, it becomes difficult to manage as the business grows.

Limitations:

- Time-consuming and labor-intensive
- High chances of human errors
- Difficult inventory tracking
- Delayed order processing
- Lack of centralized management

Traditional E-Commerce Platforms

Platforms such as Shopify, WooCommerce, and Magento provide built-in solutions for managing online stores. These platforms offer product management, order processing, and payment integration features.

Limitations:

- Limited customization options
- Subscription and maintenance costs
- Dependency on third-party services
- Restricted control over backend functionality
- Additional costs for premium features and plugins

Standalone Inventory Management Systems

Many businesses use separate inventory management software to track stock levels and product availability. These systems focus mainly on inventory control and stock monitoring.

Limitations:

- Lack of integration with order management
- Separate database management
- Increased operational complexity
- Additional software maintenance requirements

Basic Order Tracking Systems

Some organizations use simple order management applications that focus only on order processing and delivery tracking.

Limitations:

- Limited inventory control
- No integrated payment management
- Lack of detailed reporting and analytics
- Difficult to scale for growing businesses

Shortcomings of Existing Approaches

- Lack of a centralized platform for managing products, inventory, orders, and payments.
- Dependence on multiple disconnected systems for different business operations.
- Limited real-time inventory monitoring and stock updates.
- Difficulty in generating comprehensive business reports and analytics.
- Higher chances of manual errors during order and inventory management.
- Limited scalability for growing e-commerce businesses.

Approach

The project follows a four-stage development and management workflow, where each stage contributes to efficient e-commerce administration and order processing.

Step 1 — Product and Category Management

- Administrators securely log in to the system.
- Product categories are created and managed.
- New products are added with descriptions, pricing, images, and stock details.
- Existing products can be updated or removed whenever required.
- Product information is stored in the centralized database.

Output: Product database containing categories, products, pricing, and inventory details.

Step 2 — Inventory Management

- Real-time stock quantities are monitored.
- Product availability is automatically updated after successful orders.
- Inventory records are maintained in the database.
- Low-stock products can be identified quickly.
- Stock inconsistencies and overselling are prevented.

Output: Updated inventory records with accurate stock information.

Step 3 — Order and Payment Processing

- Customer orders are received and processed.
- Order details are stored securely in the database.
- Payment transactions are handled through Razorpay integration.
- Payment status is verified automatically.
- Order statuses are updated as Pending, Confirmed, Shipped, Delivered, or Cancelled.

Output: Order records, payment confirmations, and updated order status information.

Step 4 — Reporting and Administration Dashboard

- Administrators can monitor all business activities from a centralized dashboard.
- Sales reports are generated automatically.
- Revenue and order statistics are analyzed.
- Inventory performance and product sales trends are displayed.
- Business insights help improve decision-making.

Tools and Technologies

Component	Technology
Frontend	React.js
Backend	Django
API Development	Django REST Framework (DRF)
Database	PostgreSQL
Payment Gateway	Razorpay
Language	JavaScript, Python, SQL
Version Control	Git & GitHub
IDE	Visual Studio Code
API Testing	Postman
Browser	Google Chrome, Mozilla Firefox

Workflow

The end-to-end data flow of the system is as follows:

1. Administrator logs into the system.
2. Products and categories are managed through the admin dashboard.
3. Inventory information is updated and monitored in real time.
4. Customer orders are received and stored in the database.
5. Payment transactions are processed through Razorpay.
6. Order status is updated throughout the delivery process.
7. Sales and inventory reports are generated automatically.
8. Administrators monitor business performance through the dashboard.

Data Flow Summary

Admin Login → Product Management → Inventory Management → Order Processing
→ Payment Verification → Order Tracking → Reports & Analytics Dashboard

Assumptions

The system is used by authorized administrators only.

- Internet connectivity is available for payment processing.
- Product information is entered correctly by administrators.
- Razorpay payment gateway services are available and operational.
- Database services remain active during system operation.
- Orders are processed according to business requirements.
- Users have basic knowledge of web-based applications.

Implementation

Data Collection

The system collects and manages data related to products, categories, inventory records, customer orders, payment transactions, and sales reports. All information is stored in a centralized database to ensure accuracy, consistency, and efficient management of business operations.

Processing Steps

Step 1 — Authentication

Administrators securely log in to the system using their credentials. Authentication and authorization mechanisms ensure that only authorized users can access administrative functionalities and sensitive business data.

Step 2 — Product & Category Management

Administrators can add, update, and delete products and categories. Product details such as name, description, price, images, and stock information are stored and maintained in the database.

Step 3 — Inventory Management

The system continuously monitors product stock levels and updates inventory records whenever orders are processed. This helps prevent stock inconsistencies and ensures accurate product availability information.

Step 4 — Order Management

Customer orders are received, processed, and stored in the database. Administrators can update order statuses such as Pending, Confirmed, Shipped, Delivered, or Cancelled and maintain complete order history.

Step 5 — Payment Processing

The system integrates with Razorpay for secure online payment processing. Payment transactions are verified automatically, and transaction records are stored for future reference and reporting.

Step 6 — Reporting and Analytics

Sales reports, order statistics, and revenue analysis are generated using stored business data. These reports help administrators monitor performance and make informed business decisions.

Key Metrics

Metric	Description
Products Managed	Product catalog and inventory records
Orders Processed	Customer orders and order history
Payment Transactions	Razorpay payment processing
Inventory Updates	Real-time stock management
Reports Generated	Sales and performance reports
Database Support	PostgreSQL database management

Solution Design

Architecture

The E-Commerce Admin & Order Management System follows a layered architecture consisting of independent modules that work together to manage products, inventory, orders, payments, and business reports. The architecture ensures scalability, maintainability, and efficient communication between different system components.

Layer 1 — Presentation Module

- Developed using React.js
- Provides user-friendly administrative dashboard
- Manages product, inventory, order, and report interfaces
- Handles user interactions and API requests

Layer 2 — Application Module

- Developed using Django and Django REST Framework
- Handles business logic and data processing
- Manages authentication and authorization
- Processes product, inventory, order, and payment operations

Layer 3 — Database Module

- PostgreSQL database for storing application data
- Maintains products, categories, orders, inventory, and payment records
- Ensures data consistency and security

Layer 4 — Payment Integration Module

- Razorpay payment gateway integration
- Handles secure transaction processing
- Payment verification and status updates

Data Store

The system maintains data through the following database entities:

- Users — Administrator account information
- Categories — Product category records
- Products — Product details and specifications
- Inventory — Stock quantity and availability records
- Orders — Customer order information
- Order Items — Product details associated with each order
- Payments — Transaction and payment status information
- Reports — Sales and business analytics data

Key Design Decisions

- Modular architecture for easy maintenance and future scalability.
- Separation of frontend and backend using REST APIs.
- Centralized database management for all business operations.
- Real-time inventory tracking to prevent stock mismatches.
- Secure payment processing through Razorpay integration.
- Authentication and authorization mechanisms to protect administrative data.
- Scalable system design that can support future business growth.

Challenges and Opportunities

Challenges Encountered

- Managing synchronization between inventory and order processing.
- Maintaining accurate stock levels after successful order placement.
- Implementing secure payment verification through Razorpay.
- Designing efficient REST APIs for frontend-backend communication.
- Ensuring database consistency during multiple simultaneous operations.
- Testing and debugging different modules before integration.

Opportunities Identified

- Development of customer-facing online shopping functionality.
- Mobile application development for Android and iOS platforms.
- AI-based product recommendation system.
- Multi-vendor marketplace support.
- Advanced analytics and business intelligence dashboard.
- Integration with logistics and shipment tracking APIs.

Reflections on the Project

This project provided valuable practical experience in full-stack web development and e-commerce management systems. Through the development process, I gained a better understanding of frontend development, backend API creation, database management, payment gateway integration, and real-world business workflows.

Some of the key learnings from this project include:

- Understanding the importance of efficient database design for business applications.
- Learning how REST APIs enable communication between frontend and backend systems.
- Gaining practical experience in integrating third-party services such as Razorpay.
- Understanding inventory management and order processing workflows.
- Learning how reporting and analytics help businesses make better decisions.
- Developing problem-solving skills while handling real-world implementation challenges.

If I were to enhance this project further, I would focus on implementing advanced analytics, AI-based recommendations, and mobile application support.

Recommendations

Based on the development and implementation experience of the E-Commerce Admin & Order Management System, the following recommendations are suggested for organizations and businesses:

1. Maintain accurate product and inventory records to ensure smooth business operations and prevent stock mismatches.
2. Regularly monitor sales reports and order statistics to identify business trends and improve decision-making.
3. Use secure authentication mechanisms and strong passwords to protect administrative access.
4. Perform regular database backups to prevent data loss and ensure business continuity.
5. Integrate advanced analytics tools for better business insights and performance monitoring.
6. Regularly update the system to improve security, functionality, and user experience.

Outcome

The E-Commerce Admin & Order Management System successfully achieved all the planned project objectives and provides a centralized platform for managing business operations efficiently.

Objective 1 – Product Management

The system allows administrators to add, update, delete, and manage products and categories through a user-friendly interface.

Objective 2 – Inventory Management

The application provides real-time inventory tracking and automatic stock updates, reducing inventory-related errors.

Objective 3 – Order Processing

The system efficiently manages customer orders, maintains order history, and supports complete order lifecycle management.

Objective 4 – Payment Integration

Secure payment processing is achieved through Razorpay integration, ensuring reliable transaction management.

Objective 5 – Reporting and Analytics

The reporting module generates sales reports, revenue analysis, and business statistics to support decision-making.

The project demonstrates the practical implementation of modern web technologies and provides an effective solution for managing e-commerce business operations.

Enhancement Scope

The current system provides all essential administrative functionalities. However, several advanced features can be implemented in future versions.

- Development of a customer-facing online shopping portal.
- Android and iOS mobile application development.
- AI-based product recommendation system.
- Multi-vendor marketplace functionality.
- Integration with logistics and shipment tracking services.
- Advanced business intelligence and analytics dashboard.
- Email and SMS notification services.
- Discount coupon and promotional campaign management.
- Role-based access control with multiple administrative levels.
- Cloud deployment for improved scalability and availability.

Link to Code and Executable File

Source Code Repository

GitHub-Repository:

[https://github.com/Sanchita088/AIdriven_compliance_intelligence_system.git]

Project Structure

File / Folder	Description
frontend/	React.js user interface
backend/	Django backend application
users/	Authentication and user management
products/	Product and category management
orders/	Order processing and tracking
payments/	Razorpay payment integration
reports/	Sales reports and analytics
database/	PostgreSQL database
media/	Product images and uploads
api/	REST API endpoints

References

- [1] Django Documentation – <https://docs.djangoproject.com/>
- [2] React Documentation – <https://react.dev/>
- [3] Django REST Framework Documentation – <https://www.django-rest-framework.org/>
- [4] PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- [5] Razorpay Documentation – <https://razorpay.com/docs/>
- [6] MDN Web Docs – <https://developer.mozilla.org/>
- [7] Git Documentation – <https://git-scm.com/doc>
- [8] GitHub Documentation – <https://docs.github.com/>
- [9] Postman Documentation – <https://learning.postman.com/>
- [10] W3Schools Web Development Tutorials – <https://www.w3schools.com/>